



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • START THE CLUSTER • STEPS 1 AND 2

1 ☐ Create a container that is a Kubernetes cluster

```
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 rancher/k3s:v1.31.5-k3s1 server --disable=traefik
```

<code>-d</code>	Detached. It runs in the background and you get your prompt back.
<code>--name k8s</code>	A name of your choosing. Without it, Docker still identifies the container, but by its id: 8d7726f96f747554440565a92a034cda1f767726619081595bab18cfd76987 Every later command would have to carry that, or its short form 8d7726f96f74. k8s is easier to type and easier to get right.
<code>--privileged</code>	Lifts the usual restrictions. k3s mounts cgroups and runs its own container runtime inside this container.
<code>--tmpfs /run</code>	Two small writable in-memory filesystems that k3s needs (also /var/run).
<code>-p 6443:6443</code>	HOST:CONTAINER. Port 6443 on your laptop is forwarded to port 6443 inside the container. The left one is yours, the right one is the container's. 6443 is where the Kubernetes API server listens.
<code>rancher/k3s:v1.31.5-k3s1</code>	The image. rancher/k3s is the repository, v1.31.5-k3s1 is the tag. You named no registry, so Docker went to Docker Hub and fetched it from there.
<code>server</code>	Not a docker flag. Everything after the image name is the command handed to the program inside. server tells k3s to be the control plane.
<code>--disable=traefik</code>	Also k3s's, not docker's. Traefik is an ingress controller : it routes HTTP arriving from outside to services inside. It claims ports 80 and 443 and adds startup time, and nothing today arrives from outside.

2 ☐ Step inside the container you just made

```
docker exec -it k8s sh
```

`-i` keeps the input open, `-t` gives you a terminal. Together they get you an **interactive shell running inside that container** — not on your laptop. Your prompt changes; that is the sign.

Prove it before you go on. Type `hostname` and write down what it says: _____

Keep that value. It comes back in panel 4, and it is not a coincidence. `exit` leaves the shell; the container keeps running without you.

Do not close this shell.

Steps 3 to 6 and every `kubectl` in panels 6 to 10 are typed **here**, at this prompt. If you type them on your laptop instead, you will get `kubectl: command not found` — the cluster's tools live in the container, not on your machine.

Closed it by accident? `docker exec -it k8s sh` puts you straight back. Nothing is lost: the cluster never stopped.

2 • THE FIVE WORDS THE NEXT FOUR STEPS USE

Read this before step 3. Everything below happens **inside** the cluster you just started, and none of it is a program you run.

kubectl	The cluster's command-line client. It does almost nothing itself: it turns what you type into a request, sends it to the API server inside the container, and prints the reply. When you "create" something, all that happens is that the cluster writes down what you asked for.
controller	A loop that runs inside the cluster, forever. It compares what you wrote down against what actually exists , and acts on the difference. You never call it and it never finishes. This is the part that does the work.
pod	The smallest thing the cluster runs: one or more containers placed together and sharing one address. You will not create a pod . A controller creates them, because of something you wrote down.
Deployment	Something you write down that says: this many pods of this image should be running, always . Its controller creates them, and replaces any that stop — whatever the reason, including finishing successfully.
Job	Something you write down that says: run this image once, until it exits 0 . Its controller creates one pod, waits for it to succeed, and stops. Nothing is replaced.

One more, because you will see it on screen: **CrashLoopBackOff**. When a Deployment's pod keeps stopping, the controller does not retry at full speed forever. It waits 10s, then 20s, then 40s, doubling up to five minutes between attempts. The name is misleading: it does not mean the container crashed. It means **it stopped again, and I am waiting a while before the next try**.

3 • DRIVE THE CLUSTER • STEPS 3 TO 6

Type these at the shell from step 2, inside the container.

3 ☐ Ask the cluster to keep a whale running

```
kubectl create deployment whale --image=ghcr.io/oesteban/whalesay --replicas=3
```

A **Deployment** says: this many of these should exist, at all times. You do not say when to start anything.

Why is the registry spelled out here, when step 1 named none?

Because there are two copies of this image and they are **not the same thing**. Run these two *on your laptop*, not in the cluster shell:

```
docker buildx imagetools inspect ghcr.io/oesteban/whalesay
docker buildx imagetools inspect oesteban/whalesay
```

The first prints a Manifests: list with Platform: linux/amd64 **and** Platform: linux/arm64. The second prints no list at all: it is a single image, linux/amd64 only.

An Apple Silicon Mac runs an **arm64** Linux VM, and k3s's containerd has no emulation to fall back on. The Docker Hub copy would give `exec format error` and the pod would never start. The GHCR copy was built for both by GitHub Actions on Friday.

On Linux or Windows, on Intel or AMD: the Docker Hub copy runs perfectly well. Check the manifests above, then try it alongside: `kubectl create deployment hub --image=oesteban/whalesay --replicas=1` Which architecture is your laptop?

4 ☐ Look at the pods the controller made for you

```
kubectl get pods
```

 — once straight away, once about a minute later.

This lists the **pods** the Deployment created on your behalf. Two columns matter: **STATUS**, what the pod is doing right now, and **RESTARTS**, how many times the container inside it has been started again.

Take both readings in the **first two minutes**. The retries slow down as they go (see *CrashLoopBackOff* in block 2), so two late readings can show the same number and tell you nothing.

5 ☐ Ask for the same image to run once, instead of forever

```
kubectl create job whale --image=ghcr.io/oesteban/whalesay -- cowsay 'a cluster ran me'
```

A **Job** says: run this to completion, once. Same image, same cluster, same command — only the kind of object is different.

6 ☐ Read what it printed

```
kubectl logs job/whale
```

`kubectl logs` prints whatever a container wrote to its output. You did not have to find the pod: naming the Job was enough.

4 • WHAT YOU STARTED

From `kubectl get nodes`:

NAME	_____	VERSION	_____
STATUS	_____	AGE when first Ready	_____

5 • THE TWO READINGS, AND THE REASON

First reading

STATUS _____

RESTARTS _____

A minute later

STATUS _____

RESTARTS _____

Compare the NAME in panel 4 with the `hostname` you wrote in step 2.

The container did not crash. It printed its whale and exited **successfully**. So why is Kubernetes starting it again?

The Job in step 5 used **the same image** and did not do this. What did you change, in one sentence?

Then read what Kubernetes actually claims.

`kubectl describe pod <one of them>` and find the block called **Last State**.

Exit Code _____ Reason _____

Now read the message beside **State: Waiting**. It contains the words `restarting failed container`.

Was it a failure? Reconcile those two lines, in one sentence.

6 • WHAT THE LOGS WILL TELL YOU

Logs are the first place you look when something is wrong, and they are not only for things that are wrong. Try all four, inside the cluster.

`kubectl logs deploy/whale`

There are three pods. Which one did it pick, and how do you know?

`kubectl logs <pod> --previous`

The container that **already died**. Why would that ever be the only copy left?

`kubectl logs -l app=whale --prefix`

All three at once. What does the prefix tell you?

`kubectl logs <pod> --timestamps`

Now you can see **when**. How far apart are the restarts?

A pod is gone and you want to know why. Which of those four do you reach for, and what would you have lost if you had waited an hour?

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus.

Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

7 • NOW USE THE IMAGE YOU BUILT ON FRIDAY

Everything so far ran **someone else's** image. Repeat step 2 with your own, the one your GitHub Actions workflow published on Friday:

```
kubectl create deployment mine --image=ghcr.io/<your-handle>/whalesay
```

Your handle _____ Did it pull? _____

If it did not, `kubectl describe pod` will name the reason. Write it down, then fix it.

Nobody told the cluster where `ghcr.io` is, and nobody gave it a password. So: what **is** a registry, and what had to be true of your package for this to work at all?

On Friday you pushed to **GHCR**. When you type `docker pull alpine` you get an image from **Docker Hub**, and you never type a registry name. Where did that default come from, and what is the full name your `alpine` actually has?

8 • THE CONTROLLER'S OWN ACCOUNT

In panel 5 you read **Last State** from `kubectl describe pod`. The same output ends with an **Events** block. Read it, then widen to the whole cluster:

```
kubectl get events --sort-by=.lastTimestamp | tail -8
```

Nobody wrote this log. Who did, and what is it a record of — what you asked for, or what the cluster did about it?

9 • TWO IMAGE STORES, ONE NAME

Inside the cluster: `crictl images`.

On your laptop, in another terminal: `docker images`.

whalesay in `crictl`? _____ in `docker`? _____

Nobody copied anything from your laptop into the cluster. So how did the image get there?

10 • A WISH IS NOT A COMMAND

`kubectl scale deployment whale --replicas=5`, wait, then `kubectl scale deployment whale --replicas=0`. Count the pods after each.

pods at 5 _____ pods at 0 _____

Now delete the **Job's** pod: `kubectl delete pod <the job pod>`. Does it come back? Does a Deployment's pod come back? Why are the answers different?

11 · EXTRA MILE · SATISFY THE DEPLOYMENT INSTEAD OF ABANDONING IT

In step 5 you stopped the restarting by changing the **object**. There is a second way that leaves it a Deployment: same image, no rebuild, no re-push, sitting at `RESTARTS 0` and `Running`. Write the command.

You have now satisfied a Deployment two ways. Which would you use for a web server, and which for a nightly report?

No hint. The image runs one program and then stops; nothing says it has to.

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.